SM11 - Code that writes code that writes code?! Creating ADaMs in R with {admiralroche}



Gordon Miller and Edoardo Mancini

PHUSE EU Connect 2026, Glasgow, UK

Table of contents

- **Background:** Why {admiralroche}, and why ADaM Templates?
- **Stage 1:** Code - Initial intro to ADaM templates
- **Stage 2:** Code that writes code - “Abstracting away” our ADaM templates
- **Stage 3:** Code that writes code that writes code - Creating dynamic ADaM templates
- **Conclusion:** Reflections, Further work, Recommendations

AI Declaration: AI tools (Google Gemini) were used in this presentation only to generate images.

Background

The open source {admiral} framework makes it very easy to get simple ADaM template programs…

In console:

```
library(admiral)

admiral::use_ad_template(
  “adsl”,
  package = “admiral”
)
```


ad_adsl.R

Source on Save

```
# Name: ADSL
#
# Label: Subject Level Analysis Dataset
#
# Input: dm, ds, ex, ae, lb
library(admiral)
library(pharmaversesdtm) # Contains example datasets from the CDISC pilot project
library(dplyr)
library(lubridate)
library(stringr)

# Load source datasets ----

# Use e.g. haven::read_sas to read in .sas7bdat, or other suitable functions
# as needed and assign to the variables below.
# For illustration purposes read in admiral test data.

dm <- pharmaversesdtm::dm
ds <- pharmaversesdtm::ds
ex <- pharmaversesdtm::ex
ae <- pharmaversesdtm::ae
lb <- pharmaversesdtm::lb

# When SAS datasets are imported into R using haven::read_sas(), missing
# character values from SAS appear as "" characters in R, instead of appearing
# as NA values. Further details can be obtained via the following link:
# https://pharmaverse.github.io/admiral/cran-release/articles/admiral.html#handling-c


dm <- convert_blanks_to_na(dm)
ds <- convert_blanks_to_na(ds)
ex <- convert_blanks_to_na(ex)
ae <- convert_blanks_to_na(ae)
lb <- convert_blanks_to_na(lb)

# User defined functions ----
```

...but are these templates really *that close* to Roche's interpretation of the ADaM standards?

Short answer:

Not really - you should probably make your own templates.

Long answer:

Any sponsor who implements ADaM (Roche included) makes choices in the areas where ADaM is flexible, e.g.:

- Conventions to follow (imputation techniques, visit mapping, period assignment, etc.)
- Sponsor-specific variables, parameters (custom age groups, population flags, etc.)

Additionally each sponsor's templates should include:

- Custom helper code catered to interact smoothly with their chosen SCE
- Hand-picked code to interact with dataset specs (perhaps using other pharmaverse packages such as metatools, metacore xportr...)

So, we set up the {admiralroche} package to provide ADaM templates that are as close as possible to the datasets that we routinely create at Roche.

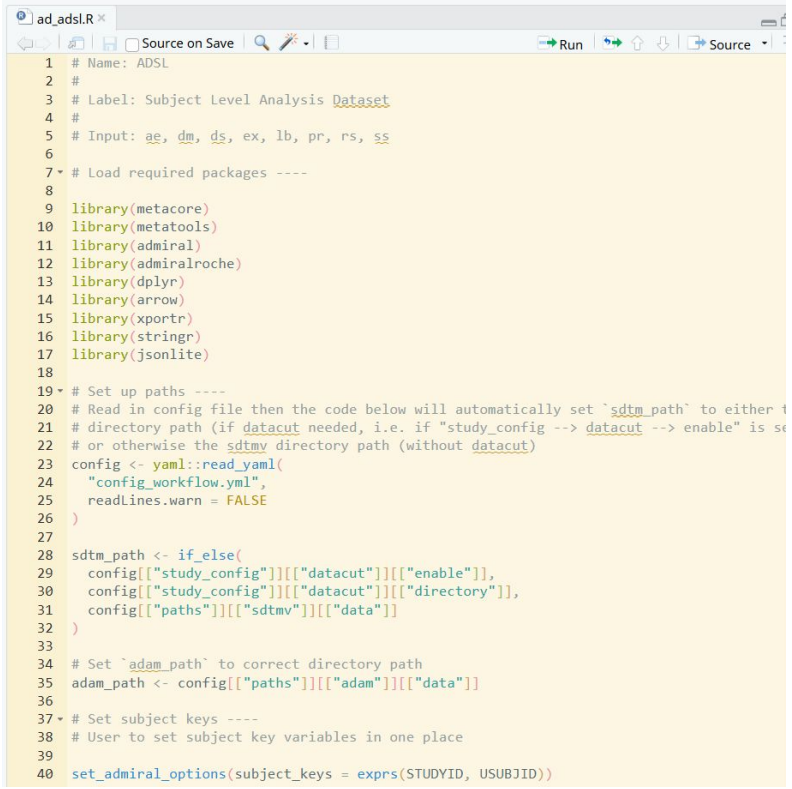
Now we can provide our own custom Roche ADaM templates!

In console:

```
library(admiral)
library(admiralroche)

admiral::use_ad_template(
  “adsl”,
  package = “admiralroche”
)
```

Note that {admiralroche} provides both the ADaM templates themselves and some R functions to derive Roche-specific variables in ADaM datasets.



Example: (portions of) an {admiralroche} template

```r
## Datacut date ----
# Get cut-off date from ATLAS activity.json meta file
activity_json <- read_json("metadatain/atlas/activity.json")
cut_date <- activity_json$activity_ccod
# pass in datacut date (in form as.Date("2020-12-31") for example)
adsl_dcut <- dm %>%
  # datacut date (DCUTDT)
  mutate(DCUTDT = as.Date(cut_date))

## Treatment ----
# See the "Visit and Period Variables" vignette to add any required phase/period variables
# (https://pharmaverse.github.io/admiral/articles/visits_periods.html)

adsl_treat <- adsl_dcut %>%
  # treatment variables (TRT01P/TRT01A)
  mutate(
    TRT01P = ARM,
    TRT01A = ACTARM
  ) %>%
  # treatment start date (TRTSDTM)
  derive_vars_merged(
    dataset_add = ex_ext,
    filter_add = (EXDOSE > 0 | (EXDOSE == 0 & str_detect(EXTRT, "PLACEBO"))) &
      !is.na(EXSTDTM),
    new_vars = exprs(TRTSDTM = EXSTDTM),
    order = exprs(EXSTDTM, EXSEQ),
    mode = "first",
    by_vars = get_admiral_option("subject_keys")
  ) %>%
...

# age group 1 (AGEGR1)
admiralroche::derive_var_agegr1() %>%
# age group 2 (AGEGR2)
admiralroche::derive_var_agegr2() %>%
```

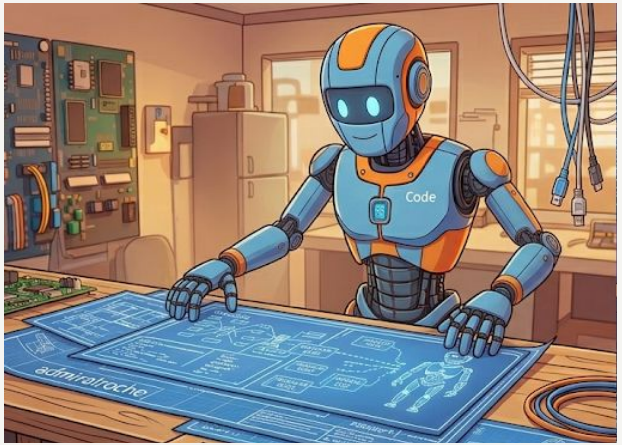
Code to pull data cut date from Roche-specific metadata file

Study-specific code to derive treatment arms (expect user to edit this)

Standard {admiral} function (possibly still using Roche-specific choice or arguments)

Custom {admiralroche} functions to derive AGEGR1 and AGEGR2 as per Roche standards

Stage 1: Code



{admiralroche}'s initial (static) ADaM templates

Throughout its first releases, {admiralroche} provided static ADaM templates, accessible through `use_ad_template(package = "admiralroche")` matching the specifications of the datasets in our Roche standards (ADSL, ADAE, ADVS, etc.)

Pros

- Simple to maintain for devs
- Simple for users to access and use
- Easiest way to initially set up {admiralroche}

Cons

- Difficult to cater for many different options, especially if complex
- Duplication of code for sections common to all templates

```
# user needs to update below derive_vars_extreme_event arguments if any other GDSR option chosen,
# e.g. by removing the sections related to RS and SS if not applicable for your study,
# or for study-specific source objects

adsl_lstalv <- adsl_dth %>%
  # last known alive date (LSTALVDT) - if beyond datacut date set to th      date
  derive_vars_extreme_event(
    by_vars = get_admiral_option("subject_keys"),
    events = list(
      roche_ds_surv, roche_ss_surv, roche_rs_date, roche_adsl_trt,
      roche_lb_date, roche_ae_start, roche_ae_end, roche_dm_death
    ),
    source_datasets = list(
      ds = ds, ss = ss, rs = rs, adsl = adsl_dth, lb = lb, ae = ae, dm = dm
    ),
    tmp_event_nr_var = event_nr,
    order = exprs(LSTALVDT, seq, event_nr),
    mode = "last",
    new_vars = exprs(LSTALVDT)
  ) %>%
```

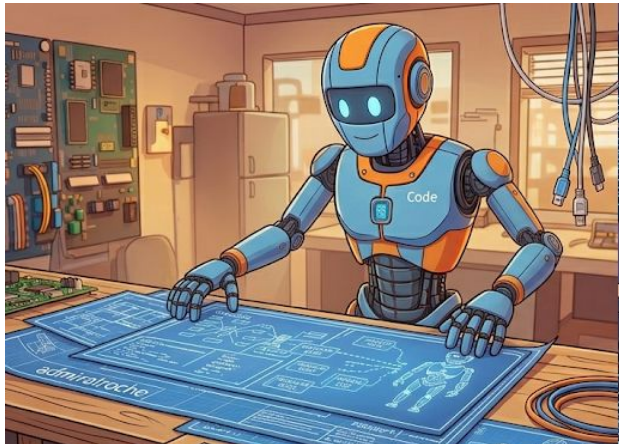
Users required to update green portions if not using most common option

```
# Check data and apply metadata
adsl <- adsl_prefinal %>%
  # Drop unspecified variables from specs
  drop_unspec_vars(metacore = metacore) %>%
  # Check all variables specified are present and no more
  check_variables(metacore = metacore) %>%
  # Checks all variables with CT only contain values within the CT
  check_ct_data(metacore = metacore) %>%
  # Orders the columns according to the spec
  order_cols(metacore = metacore) %>%
  # Sorts the rows by the sort keys
```

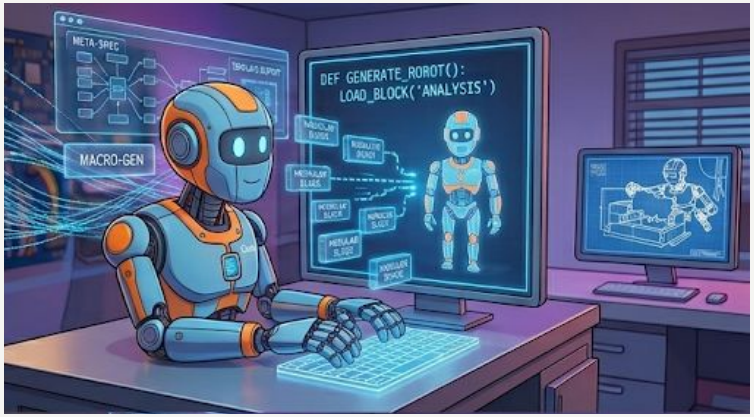
An update to this function would require updating every template

Summary: where are we at so far?

Stage	How is a typical template (e.g. ADSL) generated and served to the user?
Stage 1: *Code*	Static `ad_adsl.R` template program is directly provided by {admiralroche}



Stage 2: Code that writes code?!



Abstracting away common chunks of the templates (1/3)

😤**Issue:** After a while, {admiralroche} contained >20 ADaM templates, with many common or similar sections. Updating one of those meant potentially having to identically edit 20 files.

Eg: ADCM and ADMH templates contain almost identical sequence of 5 {admiral} code blocks to set up analysis dates, with some key differences.

For just the blue AENDTM block, we see:

- Different source variable names (`CMENDTC` vs `MHENDTC`)
- Different time imputation (`"first"` vs `"last"`)

🤔**Can we do better than just duplicating this code?** 🤔

```
## Analysis dates ----

adcm_dates <- cm %>%
  # add ADSL variables required for derivations
  derive_vars_merged(
    dataset_add = adsl,
    new_vars = adsl_derv_vars,
    by_vars = get_admiral_option("subject_keys")
  ) %>%
  # analysis start date/time (ASTDTM), imputation flag
  derive_vars_dtm(
    dtc = CMSTDTC,
    new_vars_prefix = "AST",
    highest_imputation = "M",
    ignore_seconds_flag = FALSE,
    preserve = TRUE
  ) %>%
  # analysis end date/time (AENDTM), imputation flags
  derive_vars_dtm(
    dtc = CMENDTC,
    new_vars_prefix = "AEN",
    highest_imputation = "M",
    date_imputation = "last",
    time_imputation = "last",
    ignore_seconds_flag = FALSE,
    preserve = TRUE
  ) %>%
  # analysis start/end dates (ASTDT/AENDT)
  derive_vars_dtm_to_dt(exprs(ASTDTM, AENDTM)) %>%
  # analysis start/end relative days (ASTDY/AENDY)
  derive_vars_dy(
    reference_date = TRTSDTM,
    source_vars = exprs(ASTDTM, AENDTM)
  )
```

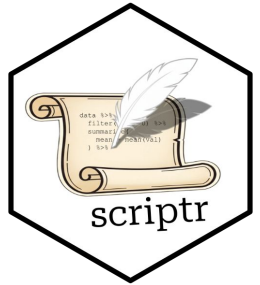
```
## Analysis dates ----

admh_dates <- mh %>%
  # add ADSL variables required for derivations
  derive_vars_merged(
    dataset_add = adsl,
    new_vars = adsl_derv_vars,
    by_vars = get_admiral_option("subject_keys")
  ) %>%
  # analysis start date/time (ASTDTM), imputation flag
  derive_vars_dtm(
    dtc = MHSTDTC,
    new_vars_prefix = "AST",
    flag_imputation = "both",
    ignore_seconds_flag = FALSE,
    preserve = TRUE
  ) %>%
  # analysis end date/time (AENDTM), imputation flags
  derive_vars_dtm(
    dtc = MHENDTC,
    new_vars_prefix = "AEN",
    highest_imputation = "M",
    date_imputation = "last",
    time_imputation = "first",
    ignore_seconds_flag = FALSE,
    preserve = TRUE
  ) %>%
  # analysis start/end dates (ASTDT/AENDT)
  derive_vars_dtm_to_dt(exprs(ASTDTM, AENDTM)) %>%
  # analysis start/end relative days (ASTDY/AENDY)
  derive_vars_dy(
    reference_date = TRTSDTM,
    source_vars = exprs(ASTDTM, AENDTM)
  )
```

Abstracting away common chunks of the templates (2/3)

💡 **Solution:** What if we abstracted away the common sections, to **dynamically build and maintain ADaM templates** using a combination of common and dataset-specific chunks?

....enter the {scriptr} package! *{scriptr} is an internal Roche package built to store code in a central place and use it to generate scripts.*



Example:

We can abstract away each block (e.g. the blue AENDTM block from before) using {scriptr}...

```
add_call_template(
  "derive_aendtm_occds",
  comment = "analysis end date/time (AENDTM), imputation flags (AENDTF/AENTMF)",
  derive_vars_dtm(
    dtc = glue_sym("{str_to_upper(domain)}ENDTC"),
    new_vars_prefix = "AEN",
    highest_imputation = "M",
    date_imputation = "last",
    time_imputation = ++time_imputation,
    ignore_seconds_flag = FALSE,
    preserve = TRUE
  )
)
```

...and then use the abstracted chunks to construct the ACDM/ADMH templates. The call below constructs the 5 ADMH blocks from the previous slide.

```
insert_calls(
  input = mh,
  ids = exprs(
    add_adsl_derv_vars, derive_astdtm_mh,
    derive_aendtm_occds(domain = "mh", time_imputation = "first"),
    derive_astdt_aendt,
    derive_study_day(
      comm = "analysis start/end relative days (ASTDY/AENDY)",
      ref_date = exprs(TRTSDTM),
      srcvars = expr(exprs(ASTDTM, AENDTM))
    )
  ),
  output = admh_dates
)
```

Abstracting away common chunks of the templates (3/3)

In summary:

- {admiralroche} was refactored internally to create a "library" of generic code chunks (powered by {scriptr}) that were used to construct our ~20 ADaM template scripts *(code that writes code!)*
- Now, any update to an {admiral}/{admiralroche} function requires **updating each chunk rather than each template**. The chunks also ensure consistency across datasets (same derivation in two datasets ⟶ same chunk used ⟶ identical code generated)

Importantly, this had no impact to our users, who could still access the templates using:

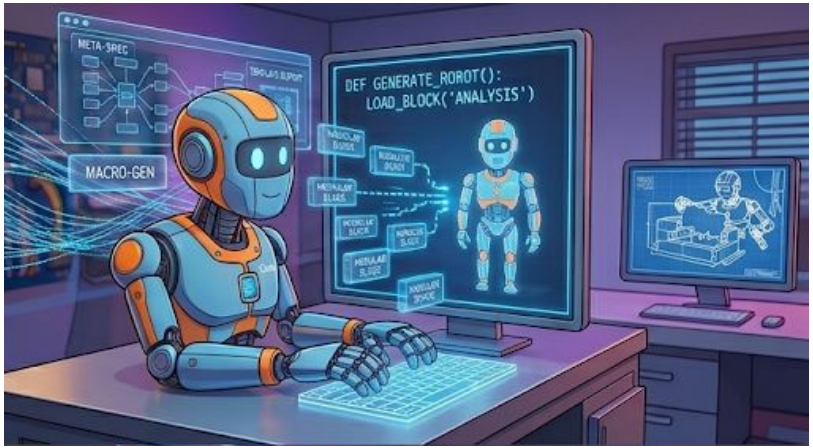
```
use_ad_template("<adam_name>", package = "admiralroche")
```

🤔**But why stop here?**🤔

Now that we had this library of code chunks, **why not just ask our users which chunks they want** (i.e. which standards they are following) to build them a template which is closer to their end goal?

Summary: where are we at so far?

Stage	How is a typical template (e.g. ADSL) generated and served to the user?
Stage 1: *Code*	`ad_adsl.R` template programs provided by {admiralroche}
Stage 2: *Code that writes code*	Internal `create_ad_adsl.R` script combines generic chunks to construct `ad_adsl.R` template - still provided to users through {admiralroche}



Stage 3: Code that writes code that writes code!



Building “dynamic” ADaM templates (1/2)

🆕 In the latest {admiralroche} releases, we:

- **Expanded our “library of chunks”** to create options and/or variants for all possible choices mentioned in our global standards - not just the most popular ones!

- Created **dynamic** `use_<adam>_template()` functions which identify user preferences through the CLI and then pick the right chunks to use to construct their ADaM templates *(code that writes code that writes code!)*
  - Users can also feed in their specifications so that the function may attempt to identify a choice by default (by scanning the derivation text and looking for keywords).

```
> use_adsl_template(dap_m3 = "my_specs.json")

i Any question can be cancelled by the escape key.
i The default value can be chosen by an empty input.
i For details on the questions see the use_adsl_template() documentation.
Which unit do you want to use for the derivation of `AAGE`?
1: years (from user-supplied DAP M3)
2: months

Selection: 1

Which reference date do you want to use for the derivation of `AAGE`?
1: RANDDT (from user-supplied DAP M3)
2: TRTSDT
3: RFICDT

<...>
Selection: 2

Which dates should be considered for deriving `LSTALVDT`?
1: survival follow up from DS
2: survival follow up from SS
3: tumor assessment (from user-supplied DAP M3)
4: treatment (from user-supplied DAP M3)
5: lab assessment (from user-supplied DAP M3)
6: AE onset (from user-supplied DAP M3)
7: AE end (from user-supplied DAP M3)
8: death (from user-supplied DAP M3)
i You can select multiple values by providing a space-separated list of
  numbers.
i Use "a" to select all values.

Selection: 3 4 5 6 7
All inputs were logged to data/metadataout/use_adsl_template.json.
Script written to /tmp/Rtmp9qhKRL/file1e35e4a8344.R
The script has been written to /tmp/Rtmp9qhKRL/file1e35e4c3b8de9.R.

Script written to programs/adam/ad_adsl.R
```

Defaults from specs

Notice ss, ds, death not required

Building “dynamic” ADaM templates (2/2)

➡️Let’s follow through the selection for LSTALVDT:

Internally, the correct chunks are identified and set up for the last alive date derivation (excluding `ds`, `ss`)…

```
# Create the required source objects for the LSTALVDT derivation
insert_chunks(
  ids = exprs(
    create_roche_rs_date, create_roche_adsl_trt, create_roche_lb_date,
    create_roche_ae_start, create_roche_ae_end
  )
)

insert_calls(
  input = adsl_dth,
  ids = exprs(
    derive_lstalvdt(
      events = exprs(
        roche_rs_date, roche_adsl_trt, roche_lb_date, roche_ae_start, roche_ae_end
      ),
      sources = exprs(
        rs = rs,
        adsl = .,
        lb = lb,
        ae = ae
      )
    ),
    cut_lstalvdt
  ),
  output = adsl_lstalv
)
```

…to end up with the correct LSTALVDT block in the ADSL template!

```
adsl_lstalv <- adsl_dth %>%
  # last known alive date (LSTALVDT)
  derive_vars_extreme_event(
    by_vars = get_admiral_option("subject_keys"),
    events = list(roche_rs_date, roche_adsl_trt, roche_lb_date, roche_ae_start, roche_ae_end),
    source_datasets = list(
      rs = rs,
      adsl = .,
      lb = lb,
      ae = ae
    ),
    tmp_event_nr_var = event_nr,
    order = exprs(LSTALVDT, seq, event_nr),
    mode = "last",
    new_vars = exprs(LSTALVDT)
  ) %>%
  # if last alive date is beyond datacut date, set to this date
  mutate(LSTALVDT = if_else(LSTALVDT > DCUTDT, DCUTDT, LSTALVDT))
```

Summary: where did we get to?

Stage	How are templates generated and served to the user?
Stage 1: *Code*	`ad_adsl.R` template programs provided by {admiralroche}
Stage 2: *Code that writes code*	Internal `create_ad_adsl.R` script combines generic chunks to construct `ad_adsl.R` template - still provided to users through {admiralroche}
Stage 3: *Code that writes code that writes code*	`use_adsl_template()` functions dynamically generates *custom* `create_adsl.R` script by choosing chunks based on user input, this custom script is automatically executed to serve *custom* `ad_adsl.R` to the user



Conclusion: Reflections and Further Work, Recommendations

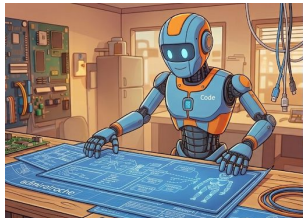
Reflections and Further Work

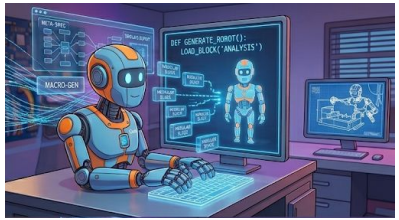
🤔 Looking back...

- It would have been **difficult to jump straight to stage 3** *(code that writes code that writes code)* because our learnings from previous stages influenced design decisions.
- The end product is a great example of a case where **deterministic automation trumps probabilistic AI**: we don't have AI guessing what the ADaM code should be, rather we have a chunk library and use it to automatically build fit-for-purpose templates.
- Initial script creation times for our users have **halved**, as has the maintenance load for {admiralroche} developers.

💡 Looking forward...

- AI agents could read specifications and make selections for users.
- Exposing statistical programmers to some of this infrastructure, allowing them to make their own chunks, to maintain study-specifics at higher level such as across-molecule or TA - maybe this is Stage 4 😮

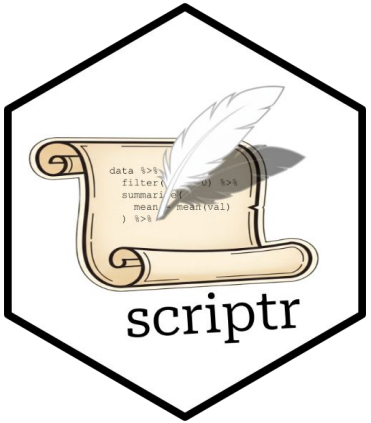






Recommendations

- You can easily set up your own {admiral<company>} package to begin with, at the very least to provide your users with templates that are a close starting point for their ADaMs.
- These templates should be allowed to **grow organically and stabilise** before you attempt to abstract further.
- Don't underestimate the importance of setting up a package like {scriptr} as the automation engine.
- A set of **mature and stable company ADaM implementation standards** is just as important as your R package.



Thank you for listening!

- Special thank you to Stefan Bundfuss for his invaluable expertise and leadership as lead developer on {admiralroche} and {scriptr}

- Thank you to Roche for allowing us to deliver this talk.

Any questions?



Contact details:

Gordon Miller
Roche Products Ltd, UK
gordon.miller@roche.com

Edoardo Mancini
Roche Products Ltd, UK
edoardo.mancini@roche.com

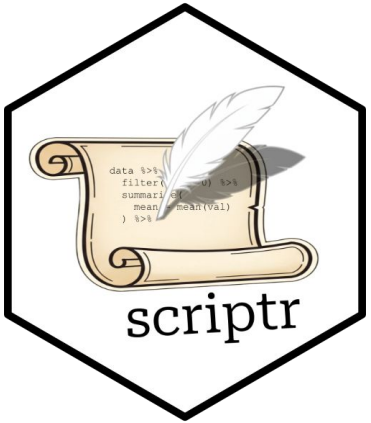
Stefan Bundfuss
Syneos Health, Germany
stefan.bundfuss@external.roche.com

Back-up slide 1: Why is {scriptr} not open-source (yet?)

There is **no direct blockage** to open-sourcing {scriptr}. However:

- When {scriptr} was in active development, it was **easier to develop it within our closed-source ecosystem** alongside {admiralroche}.
- We assessed that other sponsors may not yet be at a point where they would use {scriptr}.

If interest in the package grows, and other sponsors/individuals want to commit developer resource to the package, we would look to open-source it.



Back-up slide 2: The {admiralrochetools} split

In this presentation, for simplicity we did not mention that in actual fact, prior to Stage 3 we **split {admiralroche} into two separate packages**.

- {admiralroche} retained the Roche-specific derivation functions and their docs.
- 🆕 {admiralrochetools} was created to house all the machinery for the ADaM templates.

Why? Mainly to **slim regulatory packages** (no need to submit templates, i.e. no need to submit {admiralrochetools}).

For more details, please see our paper.



